

CSE-214 Behavioural Design Patterns

Online Lab Coding Exam Survival Pack

Based primarily on the provided course slides and previous exam questions

Contents

1	Scope	2
2	Fast Recognition Table	2
3	Strategy	2
4	Template Method	3
5	Observer	3
6	Mediator	4
7	State	4
8	Exam-derived Pattern Recognition	5
9	30-Minute Coding Workflow	5
10	Practice Problems	6
11	Mixed Pattern Questions	6
12	Final Last-Minute Cheat Sheet	7
13	How to Use the Java Templates	7

1 Scope

The syllabus patterns covered here are Strategy, Template Method, Observer, Mediator, and State. Command appeared in the slide material but is excluded from this pack because it is outside the stated syllabus.

2 Fast Recognition Table

Pattern	Recognition clue	First code you should think of
Strategy	Same job, different interchangeable algorithms; runtime switching	interface Strategy + Context has Strategy
Template Method	Same overall workflow/skeleton, some steps vary by type	abstract class + final template method
Observer	One object changes, many subscribers must automatically react	List<Observer>, register/remove/notify
Mediator	Many objects coordinate but should not know each other directly	Mediator reference in components; mediator routes events
State	Same object behaves differently depending on current condition/workflow stage	State interface; Context delegates to current State

3 Strategy

The slide defines Strategy as defining a family of algorithms, encapsulating each one, and making them interchangeable. The key exam clue is that behavior may be changed at runtime.

Mental map

```
Context --has-a--> Strategy interface
                  ^   ^   ^
                  A   B   C
```

Exam recipe

1. List the interchangeable algorithms.
2. Create one Strategy interface method representing the varying behavior.
3. Create one concrete class per algorithm.
4. Put a Strategy field inside Context.
5. Add setStrategy() if runtime switching is required.
6. Delegate the work to strategy.execute(...).

Typical question clues

Payment method, route selection, discount calculation, compression algorithm, notification channel, sorting method, travel policy.

Common mistakes

Do not put a large switch/if-else for algorithms inside Context. Do not use inheritance when the main clue is runtime replacement of a behavior.

4 Template Method

The slide describes a superclass that defines the skeleton of an algorithm while subclasses override selected steps without changing the structure.

Mental map

AbstractProcess

```
final run()
  common
  variable (abstract)
  common
  variable (abstract)
  common
  ^
  / \
TypeA TypeB
```

Exam recipe

1. Identify the fixed order.
2. Put the entire sequence in one final method.
3. Keep common steps implemented in the abstract superclass.
4. Make differing steps abstract/protected.
5. Use a hook method when an optional step depends on subclass choice.

Typical question clues

Hospital departments follow the same visit flow but use different assessment/treatment; report generation follows the same pipeline; different file formats use the same processing sequence.

Strategy vs Template Method

Template Method is inheritance/class-level variation: subclasses change selected steps. Strategy is composition/object-level variation: a context can receive another strategy at runtime.

5 Observer

The slide frames Observer as a one-to-many dependency: when a subject changes state, dependents are notified and updated automatically.

Mental map

Subject

```
List<Observer>
register/remove
notify
  |
+---+---+
A   B   C
```

Exam recipe

1. Define `Observer.update(...)`.
2. Define `Subject` `register/remove/notify`.
3. Keep `List<Observer>` in the concrete subject.
4. When important data changes, save it and call `notifyObservers()`.
5. Each observer can react differently.

Typical clues

Subscribers, widgets, dashboards, groups receiving alerts, notifications, stock displays, weather displays.

Push vs pull

In a push style, `update(...)` receives the new data. In a pull style, `update()` receives the subject/reference and asks it for data. Use whichever style the question naturally suggests; push is usually faster to code.

6 Mediator

The slide says direct communication between components should cease; components collaborate indirectly through a mediator that redirects or coordinates actions.

Mental map

```
A --\
B ----> Mediator ----> appropriate components
C --/
```

Exam recipe

1. Define `Mediator.notify(sender,event)`.
2. Give each component a `Mediator` reference.
3. Components notify the mediator instead of calling colleagues.
4. The mediator stores/references relevant components.
5. The mediator decides the resulting coordination.

Typical clues

Central Hub, control tower, dialog box, coordinator, office coordinator, smart home devices, UI controls affecting one another.

Observer vs Mediator

Observer is broadcast/subscription: one subject tells interested observers that something changed. Mediator is coordination: the central object decides how components should interact.

7 State

The slide defines State as allowing an object to alter its behavior when its internal state changes, so it appears to change class. Previous questions emphasize workflows, allowed/invalid operations, final states, and transitions.

Mental map

```
Context --> current State
           ^   ^   ^
           A   B   Final
```

Each state decides operation validity and transitions.

Exam recipe

1. Copy every required operation from the question into the State interface.
2. Create Context with a current State field.
3. Context delegates each public operation to current state.
4. Each concrete state implements every operation.
5. Valid operation: perform action and/or transition.
6. Invalid operation: print an appropriate message.
7. Final state: all transition-changing operations remain invalid.
8. Keep shared object data in Context.

Very important

Do not solve the entire workflow with giant if/else statements in Context. The behavior belongs to the concrete State classes.

Temporary state

If a question says a temporary state should later return to the previous state, Context can store previousState before replacing the current state, then restore it later.

8 Exam-derived Pattern Recognition

The provided question set includes examples that strongly indicate:

- Observer: a central board/data source with dynamically added and removed groups/widgets receiving updates.
- Mediator: a Smart Home where devices report to a Central Hub and do not communicate directly.
- Template Method: hospital departments with the same Check-In/Vitals/.../Discharge skeleton but differing assessment and treatment.
- State: subscription tiers and a return/refund lifecycle where legal operations depend on the current condition, including final states.

9 30-Minute Coding Workflow

1. Spend 2–3 minutes identifying the pattern and writing the class/interface names.
2. Write interfaces/abstract classes first.
3. Write Context/Subject/Mediator skeleton next.
4. Implement concrete classes one by one.
5. Trace one complete scenario manually.
6. Compile early. Fix naming/signature errors before adding extra output.

10 Practice Problems

1. Strategy

A navigation app can use `DrivingRoute`, `WalkingRoute`, and `CyclingRoute`. The user may change route mode while the app is running. Implement runtime switching.

2. Template Method

Three departments process a service request with the fixed order `Register`, `Verify`, `SpecializedProcessing`, `GenerateResult`, `Close`. `SpecializedProcessing` differs; one department optionally performs `ExtraReview`.

3. Observer

A university announcement board allows `Clubs`, `Departments`, and `Students` to subscribe/unsubscribe. When an announcement is posted, all current subscribers react.

4. Mediator

A smart home has `LightSensor`, `Blinds`, `AirConditioner`, and `CentralHub`. Devices report events only to the hub. The hub decides which devices react.

5. State

A document starts `Draft`. It may be `Submit`, `Approve`, `Reject`, or `Cancel` depending on the current state. `Approved`, `Rejected`, and `Cancelled` are final.

11 Mixed Pattern Questions

A. Identify the pattern

“A delivery app lets users switch between bicycle, bike, and car cost-calculation algorithms while an order object remains the same.” Strategy.

B. Identify the pattern

“Every data importer performs `Open`, `Parse`, `Validate`, `Store`, `Close`; only `Parse` and `Validate` differ by format.” Template Method.

C. Identify the pattern

“A stock data source updates any number of display widgets, which can be attached and detached.” Observer.

D. Identify the pattern

“A dialog coordinates text fields, checkboxes, and buttons so controls do not directly depend on one another.” Mediator.

E. Identify the pattern

“A request allows cancel in `Requested` and `Approved` but not after `Delivered`; final conditions reject every later action.” State.

12 Final Last-Minute Cheat Sheet

STRATEGY

```
interface Strategy { action(); }  
Context has Strategy  
setStrategy(new X())
```

TEMPLATE METHOD

```
abstract class  
final process() = fixed order  
abstract/protected varying steps
```

OBSERVER

```
List<Observer>  
register/remove/notify  
state change -> notifyObservers()
```

MEDIATOR

```
Component has Mediator  
component -> mediator.notify(this,event)  
mediator coordinates components
```

STATE

```
interface State = ALL question operations  
Context has current State  
Context delegates  
State has Context  
transition: context.setState(new NextState(context))  
invalid action: message  
final state: nothing changes
```

13 How to Use the Java Templates

Keep the five Java files locally. During an exam, choose the matching template, rename domain classes and operations, delete unused parts, and add concrete classes/states/observers as required. The architecture should remain intact unless the question explicitly requires a different interface.